

00_frontmatter

Helios — dbt Engineering Guide

`DBT_GUIDE.md` · Companion to `HELIOS_PROJECT_BIBLE.md` §15–§17 (Analytics Engineering, dbt, BigQuery) and §8 (Data Model) · **Version:** v1.0 · **Date:** 2026-06-03

Purpose. This is the production-grade analytics-engineering handbook for Helios — the standalone reference for the dbt layer that transforms raw GA4 events into governed marts. Those marts are the foundation of the entire product: the semantic layer (`models/semantic/semantic_layer.yaml`) exposes them as metrics, and the agents compose those metrics via `semantic-mcp` — they never hand-write SQL. Correct, tested, documented, fresh marts are therefore non-negotiable. It is written assuming Helios will eventually run **production-grade and multi-tenant**, while staying honest about the static public sample dataset it is currently built on.

How to use this guide

- Build in dependency order (`DEPENDENCY_MAP.md` M1–M5): sources → staging → intermediate (the two keystones) → marts → semantic layer.
- This guide is the *how* of the dbt layer; `CLAUDE.md` is the operating rules, the Bible is the *why*, `METRIC_GOVERNANCE_GUIDE.md` owns the metric definitions downstream of these marts.
- Every code block is real and copy-usable. The keystones (sessionization, `reached_*` monotonicity, revenue reconciliation) fail **silently** — write their golden tests first (§6).

Conventions cheat-sheet

Area	Rule
Layers / prefixes	<code>stg_<source>__<entity></code> → <code>int_<source>__<entity></code> → <code>fct_*</code> / <code>dim_*</code> ; one model per file; no cross- or upward-layer refs; snake_case
Materializations	staging = <code>view</code> , intermediate = <code>ephemeral</code> , marts/core = <code>incremental</code> (<code>insert_overwrite</code>), finance/growth/dims = <code>table</code> , semantic = <code>view</code>

Area	Rule
Partition / cluster	core facts: <code>partition_by event_date (DATE, day) + cluster_by [device_category, channel_group]</code> ; <code>require_partition_filter</code> on large facts
Incremental	<code>insert_overwrite</code> + 3-day <code>is_incremental()</code> lookback (re-materializes recent partitions for late shards)
Session key	<code>session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))</code> ; <code>sessions = COUNT(DISTINCT session_key)</code>
Funnel flags	<code>reached_*</code> are max-downstream monotonic → <code>sessions ≥ reached_view_item ≥ ... ≥ reached_purchase</code> ; step rates ≤ 1 by construction (<code>did_*</code> retired)
Engaged session	<code>session_engaged = '1'</code> OR <code>engagement_time_msec ≥ 10000</code>
Channel grouping	exactly 10 groups, defined in one macro <code>channel_group_case()</code> ; <code>traffic_source</code> is user first-touch, so prefer session-scoped <code>event_params</code> source/medium
Money / rates	<code>*_in_usd</code> columns only; rates as <code>SUM(num)/SUM(den)</code> after grouping (Simpson's-paradox defense)
Marts shape	wide (denormalized with descriptive dims) so the semantic layer slices without runtime joins

Table of Contents

1. dbt Project Structure
2. Source Models
3. Staging Models
4. Intermediate Models
5. Marts
6. Testing Strategy
7. Freshness Strategy
8. Lineage Strategy
9. Documentation Strategy
10. Production-Readiness Checklist